॥ ॐ नमो भगवते वासुदेवाय नमः ॥

ATDL ESE Paper — 60M

Q2)(a) TL in content of autonomous vehicle domain — 3M

Ans)(a) TL is a DL technique where a model trained on a large dataset is reused for a different but related task. Instead of training a model from scratch, a pre-trained neural network (like AlexNet, VGGNet or ResNet) is adapted & fine-tuned for a new problem.

In the autonomous vehicle domain, transfer learning is widely used in CV tasks such as:

- Object detection
- Traffic sign recognition
- Lane detection
- Obstacle detection

A model initially trained on a large dataset like ImageNet learns general features such as edges, shapes & textures. These learned features are then transferred & fine-tuned using driving datasets to recognize road objects & environments.

TL enables faster & more efficient development of DL models used in autonomous vehicle perception systems.

M T W T F S S
Page No.: 2
Date: YOUVA

—— 4M

Q2)(b) Explain the role of Skip Connections in residual networks.

Ans2)(b) Skip Connections (also called Short cut Connections) are a key component of Residual Networks (ResNet).

→ They allow the input of a layer to bypass one or more intermediate layers & can be added directly to the output of a deeper layer.

→ Mathematically, instead of learning a direct mapping $H(x)$, a residual network learns a residual function:

$$H(x) = F(x) + x$$

Here,

- x = input
- $F(x)$ = output of stacked convolutional layer
- $H(x)$ = final output after adding the shortcut connection.

→ It solves vanishing gradient problem. In very deep neural networks, gradient may become extremely small during backpropagation, preventing effective learning. Skip connections provide a direct path for gradient flow, allowing gradients to propagate through many layers.

→ Enables training of very deep networks. Without skip connections, deep networks suffer from degradation problems (accuracy decreases as depth increases). Residual ~~Networks~~ Learning allows networks with 50, 101 or even 152 layers to train effectively.

Q2)(b) Why is YOLO faster than R-CNN? Describe: —4M

Ans2)(b) Yolo performs object detection in a single neural network pass, while R-CNN uses multiple stages to detect objects.

→ Single Stage Detection (YOLO)

Yolo treats object detection as a single regression problem. It directly predicts:

- Bounding Box coordinates.
- Object class probabilities.

from the entire image in one forward pass of the CNN. This makes detection extremely fast & suitable for real time applications.

→ Multi Stage Detection in R-CNN

R-CNN performs object detection in multiple steps:

- Region Proposal Generation (Selective Search)
- Feature Extraction using CNN for each region
- Classification of each region.
- Bounding Box refinement

Since the CNN must run separately on thousands of region proposals, the process becomes slow.

- Yolo can process 30 – 60 frames per second
- Yolo divides image into a grid (7X7). This allows Yolo to detect multiple objects simultaneously.

Q2)(C) · YOLO Object detection in detail — 8M

Ans2)(C) YOLO (You Only Look Once) is a real time object detection algorithm that detects objects in an image using a convolutional neural network (CNN).

YOLO processes the entire image at once, making it very fast.

1) Input Image Processing

- The input image is first resized to a fixed dimension & passed into a Convolutional Neural Network.
- CNN extracts important features such as edges, shapes, textures & object patterns.
- These features help the network recognize objects present in the image.

2) Dividing the image into grid cells

- YOLO divides the image into an S X S grid (commonly 7X7).
- Each grid cell is responsible for detecting objects whose center lies inside that cell.
- Each grid cell predicts bounding boxes, confidence scores & class probabilities.

3) Bounding Box Prediction

- Each grid cell predicts multiple bounding boxes.

- Each bounding box contains:
  - x → center of box (horizontal position)
  - y → center of box (vertical position)
  - h → height of box
  - w → width of box
  - confidence score

Confidence score indicates probability that an object exists and accuracy of the bounding box.

4) Class Probability Prediction

- Each grid cell also predicts probabilities for different object classes such as car, person, bicycle & traffic light.

The final class score is computed as:

Class Score = Confidence × Class Probability

This determines which object is detected.

5) Bounding Box Filtering

- Many bounding boxes may overlap. To remove duplicates, YOLO applies Non-Maximum Suppression (NMS). This ensures that only the best bounding box remains for each object.

6) Final Output

- The final output contains detected object class, bounding box coordinates & confidence score. Each object is displayed with a bounding rectangle around it.

Hidden State – Memory of network – passes data
Previous steps → next step – sequential data understand

Page No.: 6 YOUVA
Date:

Q2)(c) Anchor Box & Bounding Box ——— 8M

Ans2)(c) (Q3)(a) Purpose of hidden state in an RNN? — 3M
(Q3)(b) Significance of " " " ——— 4M

Ans3)(a) Hidden state in an RNN acts as the memory of the network. It stores information from previous time steps & passes it to the next step so the model can understand sequential data.

- It helps the model understand relationships between elements in a ~~space~~ sequence, such as words in a sentence or frames in a video.
- At each time step, hidden state is updated using the current input & previous hidden state, enabling the network to maintain context.
- This Memory allows RNNs to perform tasks where previous data affects future predictions.

Q3)(b) In what ways LSTM are different from traditional RNNs?

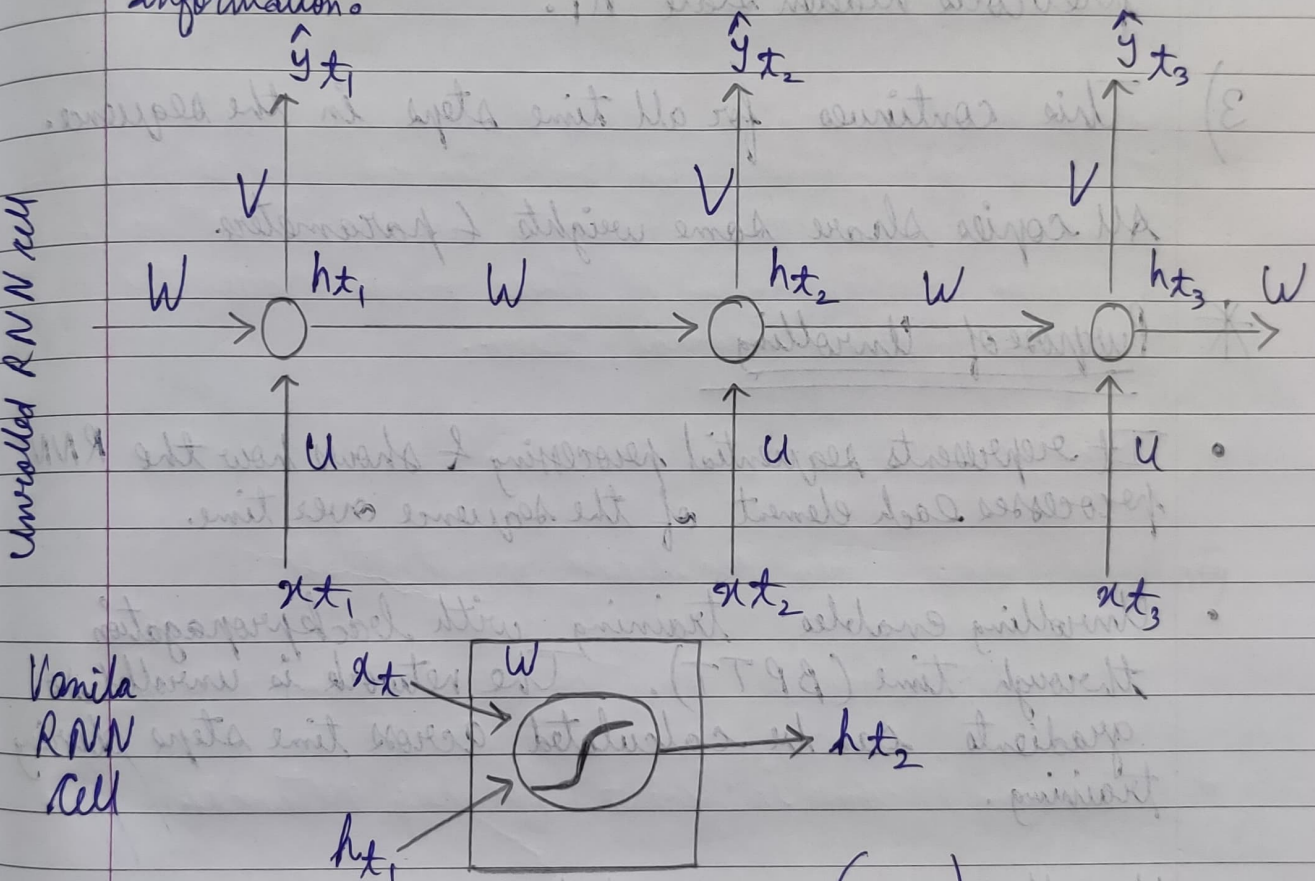
Ans3)(b)

Feature	Traditional RNN	LSTM
1) Memory	Simple Hidden State	Memory cell
2) Gradient Problem	Suffers from vanishing gradients	Solves vanishing gradients
3) Architecture	Simple	Complex with input, forget & output gates
4) Long Term Learning	Weak	Strong
5) Performance	Lower for long sequences	Better for sequential tasks

Q3)(c) Explain Concept of "unrolling" in context of RNNs — 3M
Q3)(a) RNN unit with diagram — 3M

Ans3)(c) Unrolling in a RNN refers to expanding the recurrent network across time steps to show how the same network processes sequential data step-by-step.

In an RNN, the same neural network is repeated for each time step & the hidden state is passed from one step to next, allowing the network to remember from previous information.



$$h_t = \tanh W \begin{pmatrix} x_{t_2} \\ h_{t_1} \end{pmatrix}$$

- W = Recurrent weight matrix
- h_t = Hidden Vector (Hidden State) at time t.
- x_t = input Vector at time t
- tanh = Hyperbolic tangent fn (Activation fn)
- $\hat{y}_t$ = predicted output at time t.
- V = output weight matrix
- U = Input Weight Matrix

* How unrolling works

When an input sequence is given (for example $x_1, x_2, x_3, \ldots\ldots, x_t$):

1) At time step t_1, the network receives the input x_1 & produced a hidden state h_1.

2) At time step t_2, the network receives input x_2 & previous hidden state h_1.

3) This continues for all time steps in the sequence.

All copies share same weights & parameters.

* Purpose of Unrolling

- It represents sequential processing & shows how the RNN processes each element of the sequence over time.
- Unrolling enables training with backpropagation through time (BPTT). The network is unrolled so gradients can be calculated across time steps during training.
- Unrolling maintains content across sequence as hidden states connect each step, allowing the model to retain information from earlier inputs.

→ Unrolling in RNNs is the process of expanding the recurrent network across time steps so that sequential inputs can be processed step-by-step while sharing the same parameters. This allows models to learn temporal dependencies using BPTT.

Q3)(c) Some applications of RNNs? ——— 8M

Ans3)(c) (1) Natural Language Processing (NLP): Used for tasks such as language translation, text generation & sentiment analysis.

(2) Speech Recognition: Converts spoken language into text by analyzing sequential audio analysis.

(3) Time Series Prediction: Used for forecasting stock prices, weather prediction & sales forecasting.

(4) Machine Translation: Translates text from one language to another.

(5) Text Generation & Autocomplete: Predicts the next word in a sentence for applications like chatbots & predictive typing.

(6) Video Analysis: Processes sequences of frames for activity recognition or video captioning.

(7) RNNs are designed to process sequential data, where current inputs depend on previous information. They are widely used in tasks involving time series, language & sequence prediction.

(8) RNNs are mainly used in applications that involve sequential or time-dependent data, where information from previous inputs influence future predictions.

* Vanishing Gradient Problem :

The vanishing gradient problem occurs during the training of deep neural networks (RNNs) when the gradient becomes extremely small during backpropagation. As a result, the weights of earlier layers update very slowly or stop updating, preventing the network from learning effectively.

How it happens?

During backpropagation, gradients are propagated backward through many layers. When activation fns such as sigmoid or tanh are used, their derivates are small (between 0 & 1). Repeated multiplication of these values causes the gradients to shrink towards zero.

Effects?

- Early layers learn very slowly or not at all
- The network fails to capture long-term dependencies
- Training becomes inefficient for deep networks.

Example

In RNNs, when processing long sequences, information from earlier time steps may dissapear because gradients become very small while propagating through many time steps.

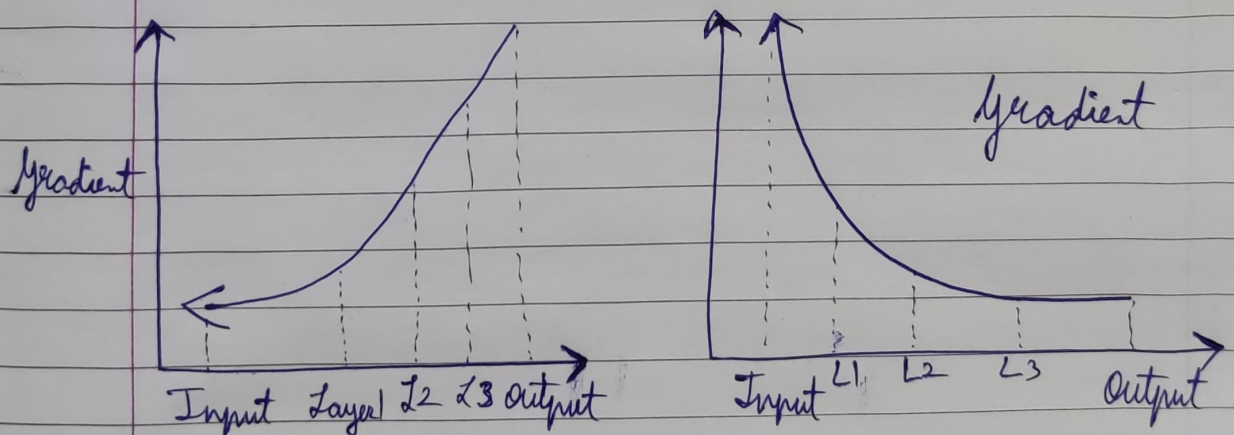
Solutions

- LSTM (Long Short Term Memory)
- GRU (Gated Reccurrent Units)
- ReLU Activation fn.
- Residual / Skip connections.

→ The vanishing gradient problem occurs when gradients becomes very small during backpropagation, making it difficult for deep neural networks to learn long term patterns.

1) Vanishing Gradient $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 < 1$

2) Exploding Gradient $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 > 1$



Vanishing Gradient Exploding Gradient